图论

1. 基础定义 2
    1.1. 图的存储 2
2. 图论 DFS 2
3. 树上问题 2
    3.1. 树的直径 2
    3.2. 最近公共祖先(Lowest Common Ancestor) 4
        3.2.1. 倍增求 lca 4
        3.2.2. dfn 求 lca 5
        3.2.3. tarjan 求 lca 7
        3.2.4. 树剖求 lca 8
        3.2.5. 欧拉序求 lca 9
    3.3. 树链剖分 10
    3.4. 虚树 10
4. 最短路 11
    4.1. bellmon flod 11
    4.2. dijstra 11
    4.3. spfa 13
    4.4. floyd 13
5. 欧拉图 13
    5.1. 定义 13
    5.2. 性质 13
6. 图的匹配 13
    6.1.1. 定义 13
    6.1.2. 二分图的判定 14
    6.1.3. 二分图的最大匹配 15
        6.1.3.1. 匈牙利算法 15
        6.1.3.2. hk 算法 16
    6.1.4. 二分图最小点覆盖(König 定理) 16
7. 联通性 16
    7.1. 点双/边双 16
        7.1.1. 无向图 16
            7.1.1.1. tarjan 求割点 16
        7.1.2. 有向图 17
    7.2. 强连通分量 17
8. 网络流 20
    8.1. 最大流 20
        8.1.1. Edmonds-Karp 算法 20
        8.1.2. Dinic 算法 22
        8.1.3. ISAP 算法 24

8.1.4. 模型 24
8.2. 费用流 24
8.2.1. SSP(Successive Shortest Path)算法 24
9. 同余最短路 25

1. 基础定义

1.1. 图的存储

链式前向星

```
void add(int a, int b) // a->b
{
    e[idx] = b, ne[idx] = h[a], h[a] = idx++;
}
for (int i = h[u]; i != -1; i = ne[i])//遍历
{
    int to = e[i];
    dfs(to);
}
```

2. 图论 DFS

- dfs 序

树 dfs 时经过节点的顺序,就是 dfs 序(dfn,是节点在 dfs 序中的编号)

性质:

祖先一定出现在后代之前

- 欧拉序

在 dfs 序的基础上,每一个子节点访问后都要记录自己

性质:

1.节点 x 第一次出现与最后一次出现之间的所有节点都在 x 的子树中

2.任意两个节点的 LCA 是欧拉序中两节点第一次出现位置中深度最小的节点

3. 树上问题

3.1. 树的直径

树上两点的最长距离

1.dfs

定理：在一棵树上，从任意节点 y 开始进行一次 DFS,到达的距离其最远的节点 z 必为直径的一端。

若存在负权边，则无法使用两次 DFS 的方式求解直径。

```
int dis, to;//直径,点
vector<int> edge[N];
auto dfs = [&](auto &dfs, int u, int fa, int d) -> void
  {
      vis[u] = 1;
      if (d > dis)
          dis = d, to = u;
      for (auto to : e[u])
          if (to != fa)
              dfs(dfs, to, u, d + 1);
  };
dis = 1;
dfs(dfs, id, 0, 1);
dis = 1;
dfs(dfs, to, 0, 1);
//最后 dis 就是树的直径
```

2.树形 dp

```
int n, d;
vector<int> edge[N];
int d1[N], d2[N];
void dfs(int u, int fa)
{
    d1[u] = d2[u] = 0;
    for (auto to : edge[u])
        if (to != fa)
        {
            dfs(to, u);
            int t = d1[to] + 1;
            if (t > d1[u])
                d2[u] = d1[u], d1[u] = t;
            else if (t > d2[u])
                d2[u] = t;
        }
    d = max(d, d1[u] + d2[u]);
```

```
}
void solve()
{
    cin >> n;
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        edge[u].push_back(v), edge[v].push_back(u);
    }
    dfs(1, 0);
    cout << d;
}
```

3.2. 最近公共祖先(Lowest Common Ancestor)

3.2.1. 倍增求 lca

时间复杂度:

预处理:$O(n \log n)$

询问:$O(\log n)$

https://www.luogu.com.cn/problem/P3379

```
void solve() {
    int n, m, s;
    cin >> n >> m >> s;
    vector<vector<int>> f(21, vector<int>(n + 1, 0));
    vector<vector<int>> e(n + 1);
    vector<int> dist(n + 1, 0);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v), e[v].push_back(u);
    }
    auto dfs = [&](auto &dfs, int u, int fa) -> void {
        f[0][u] = fa;
        for (auto &to : e[u])
            if (to != fa) {
                dist[to] = dist[u] + 1;
```

```
                dfs(dfs, to, u);
            }
    };
    dfs(dfs, s, 0);
    for (int i = 1; i <= 19; i++)
        for (int j = 1; j <= n; j++)
            if (f[i - 1][j])
                f[i][j] = f[i - 1][f[i - 1][j]];
    auto lca = [&](int u, int v) -> int {
        if (dist[u] < dist[v])
            swap(u, v);
        int d = dist[u] - dist[v];
        for (int j = 0; j <= 19 && d; j++, d /= 2)
            if (d & 1)
                u = f[j][u];
        if (u != v) {
            for (int j = 19; j >= 0; j--)
                if (f[j][u] != f[j][v])
                    u = f[j][u], v = f[j][v];
            return f[0][u];
        }
        return u;
    };
    for (int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        cout << lca(u, v) << endl;
    }
}
```

3.2.2. dfn 求 lca

时间复杂度:

预处理:$O(n\log n)$

询问:$O(1)$

查询[dfn[x]+1,dfn[y]]内深度最小的节点的父亲,就是 x,y 的 lca

比较时，取时间戳较小的结点也是正确的，不用记录深度

因为询问区间的 lca 的 dfn 一定小于区间内所有节点的 dfn

```cpp
int f[20][N];
void solve()
{
    int n, q, s, cnt = 0;
    cin >> n >> q >> s;
    vector<vector<int>> e(n + 1);
    vector<int> dfn(n + 1, 0);
    auto get = [&](int x, int y) -> int
    { return dfn[x] < dfn[y] ? x : y; };
    auto dfs = [&](auto &dfs, int u, int fa) -> void
    {
        f[0][dfn[u] = ++cnt] = fa;
        for (auto to : e[u])
            if (to != fa)
                dfs(dfs, to, u);
    };
    auto lca = [&](int u, int v) -> int
    {
        if (u == v)
            return u;
        if ((u = dfn[u]) > (v = dfn[v]))
            swap(u, v);
        int d = __lg(v - u++);
        return get(f[d][u], f[d][v - (1 << d) + 1]);
    };
    for (int i = 1; i < n; i++)
    {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v), e[v].push_back(u);
    }
    dfs(dfs, s, 0);
    for (int i = 1; i <= __lg(n); i++)
        for (int j = 1; j + (1 << i) - 1 <= n; j++)
            f[i][j] = get(f[i - 1][j], f[i - 1][j + (1 << i - 1)]);
    for (int i = 1; i <= q; i++)
    {
        int u, v;
        cin >> u >> v;
```

```
        cout << lca(u, v) << endl;
    }
}
```

3.2.3. tarjan 求 lca

预处理:$O(n)$

询问:$O(mA(m+n,n)+n)$

A 是阿克曼函数的反函数,很小,视为常数,10^80 才为 4,2047 时为 3

离线处理所有询问

把点分成三类:

v=0:未访问过的点

v=1:访问过一次的点

v=2:访问过,并且已经回溯的点

把所有询问离线下来,挂到对应的节点上形成一个二元组{to,i}

只有当 v[to]=2 时,此时 lca 就是 to 所在并查集指向的节点

注意特判 u=v 的情况,tarjan 无法处理这种情况

```
auto tarjan = [&](auto &tarjan, int u) -> void
{
    v[u] = 1;
    for (auto to : e[u])
    {
        if (v[to])
            continue;
        tarjan(tarjan, to);
        p[to] = u;
    }
    for (auto [to, i] : q[u])
    {
        if (v[to] == 2)
            ans[i] = find(find, to);
    }
    v[u] = 2;
};
```

3.2.4. 树剖求 lca

时间复杂度:预处理$O(n)$,单次询问$O(\log n)$

```
#include <bits/stdc++.h>
#define endl '\n'
#define x first
#define y second
using namespace std;
using ll = long long;
using pll = pair<ll, ll>;
const ll mod = 1e9 + 7;
void solve() {
    int n, m, s;
    cin >> n >> m >> s;
    vector<vector<int>> e(n + 1);
    vector<int> fa(n + 1), son(n + 1), top(n + 1), sz(n + 1), dep(n + 1);
    for (int i = 1; i < n; i++) {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v), e[v].push_back(u);
    }
    auto lca = [&](int u, int v) -> int {
        while (top[u] != top[v]) {
            if (dep[top[u]] > dep[top[v]])
                u = fa[top[u]];
            else
                v = fa[top[v]];
        }
        return dep[u] < dep[v] ? u : v;
    };
    auto dfs1 = [&](auto &dfs1, int u) -> void {
        sz[u] = 1;
        dep[u] = dep[fa[u]] + 1;
        for (auto &to : e[u])
            if (to != fa[u]) {
                fa[to] = u;
                dfs1(dfs1, to);
```

```
                sz[u] += sz[to];
                if (sz[to] > sz[son[u]])
                    son[u] = to;
            }
    };
    auto dfs2 = [&](auto &dfs2, int u, int h) -> void {
        top[u] = h;
        if (son[u])
            dfs2(dfs2, son[u], h);
        for (auto &to : e[u]) {
            if (to == fa[u] || to == son[u])
                continue;
            dfs2(dfs2, to, to);
        }
    };
    dfs1(dfs1, s);
    dfs2(dfs2, s, s);
    for (int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        cout << lca(u, v) << endl;
    }
}
int main() {
#ifdef ix3ea
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

3.2.5. 欧拉序求 lca

完全被 dfn lca 吊打,不用学

3.3. 树链剖分

将树剖分为若干条链,利用数据结构维护每条链

树上每经过一条轻边,子树尺寸缩小一半

剖分出的链的数量:$\frac{n+1}{2}$

剖分后,任意一条从根到叶子的路径,最多经过$\log n$条轻/重边

```
auto lca = [&](int x, int y) -> int {
    while (dep[top[x]] != dep[top[y]]) {
        if (dep[top[x]] > dep[top[y]])
            x = top[x];
        else
            y = top[y];
    }
    return x;
};
```

3.4. 虚树

二次排序+lca

时间复杂度:$O(m\log n)$m 为关键点数,n 为总点数

```
auto build = [&]() -> void
{
    sort(a.begin() + 1, a.begin() + num + 1, [&](int a, int b)
            { return dfn[a] < dfn[b]; });//将关键点按照 dfn 排序
    c.push_back(1);
    for (int i = 1; i < num; i++)
    {
        c.push_back(a[i]);
        c.push_back(lca(a[i], a[i + 1]));
    }
    c.push_back(a[num]);
    sort(c.begin() + 1, c.end(), [&](int a, int b)
            { return dfn[a] < dfn[b]; });//虚树上的所有点按 dfn 排序
    c.erase(unique(c.begin(), c.end()), c.end());//去重
    for (int i = 0; i + 1 < c.size(); i++)
    {
        ll lc = lca(c[i], c[i + 1]);
```

```
        ll d = cal(lc, c[i + 1]);
        ne[lc].push_back({c[i + 1], d});//;连边构建虚树
        ne[c[i + 1]].push_back({lc, d});
    }
};
```

4. 最短路

4.1. bellmon flod

4.2. dijstra

只适用与非负权图!

- 朴素版

时间复杂度:$O(\log n^2 + m)$

```
struct node
{
    int y, v;
    node(int _y, int _v)
    {
        y = _y;
        v = _v;
    }
};
bool used[N];
vector<node> edge[M];
int n, m, k, dist[N];
int dijkstra(int s, int t)
{
    memset(used, false, sizeof used);
    memset(dist, 127, sizeof dist);
    dist[s] = 0;
    while (true)
    {
        int x = -1;
        for (int i = 1; i <= n; i++)
            if (!used[i] && dist[i] < 1 << 30)
                if (x == -1 || dist[i] < dist[x])
                    x = i;
```

```
        if (x == t || x == -1)
            break;
        used[x] = true;
        for (auto i : edge[x])
            dist[i.y] = min(dist[i.y], dist[x] + i.v);
    }
    if (dist[t] < 1 << 30)
        return dist[t];
    return -1;
}
```

- 堆优化

时间复杂度:$O((n+m)\log m)$

```
vector<ll> dis(n + 1, 1e9);
auto dij = [&](int s) -> void
{
    vector<bool> vis(n + 1, 0);
    priority_queue<pll, vector<pll>, greater<pll>> q;
    dis[s] = 0;
    q.push({dis[s], s});
    while (q.size())
    {
        auto [dist, u] = q.top();
        q.pop();
        if (vis[u])
            continue;
        vis[u] = 1;
        for (auto [to, w] : e[u])
            if (dis[to] > dis[u] + w)
            {
                dis[to] = dis[u] + w;
                q.push({dis[to], to});
            }
    }
};
```

4.3. spfa

4.4. floyd

任意一种 ijk 顺序跑三次一定就是对的

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
    {
        dist[i][j] = 1e18;
        if (i == j)
            dist[i][j] = 0;
    }
for (int k = 1; k <= n; k++)
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++)
            if (dist[i][k] < 1e18 && dist[k][j] < 1e18)
        dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
```

5. 欧拉图

5.1. 定义

欧拉回路:通过图中每条边恰好一次的回路

欧拉通路:通过图中每条边恰好一次的通路

欧拉图:有欧拉回路的图

半欧拉图:有欧拉通路但不具有欧拉回路的图

5.2. 性质

最大独立集

6. 图的匹配

6.1.1. 定义

- 匹配

一组两两没有公共点的边集

- 二分图

节点由两个集合组成，且两个集合内部没有边的图

6.1.2. 二分图的判定

1.黑白染色

https://codeforces.com/contest/741/problem/C

有 2n 个人围成一圈坐在桌子边上,每个人占据一个位子,对应这 2n 个人是 n 对情侣

要求情侣不能吃同一种食物,并且桌子上相邻的三个人的食物必须有两个人是不同的,只有两种食物(1 或者是 2)

问一种可行分配方式。

构造二分图,染色输出方案

```
void solve() {
    int n;
    cin >> n;
    vector<vector<int>> e(2 * n + 1);
    vector<pii> p(n + 1);
    vector<int> c(2 * n + 1, 0);
    for (int i = 1; i <= n; i++) {
        auto &[u, v] = p[i];
        cin >> u >> v;
        e[u].push_back(v), e[v].push_back(u);
    }
    for (int i = 1; i <= n; i++) {
        e[2 * i].push_back(2 * i - 1);
        e[2 * i - 1].push_back(2 * i);
    }
    auto dfs = [&](auto &dfs, int u, int tag) -> void {
        c[u] = tag;
        for (auto to : e[u])
            if (!c[to])
                dfs(dfs, to, 3 - tag);
    };
    for (int i = 1; i <= 2 * n; i++)
        if (!c[i])
            dfs(dfs, i, 1);
    for (int i = 1; i <= n; i++) {
        auto &[u, v] = p[i];
        cout << c[u] << " " << c[v] << endl;
```

```
    }
    cout << endl;
}
```

2.拓展域并查集

把一个点拆成两个颜色表示

维护信息,如果一个点的两个颜色在同一个几何里,就说明不是一个二分图

这种做法适合动态加边的判断二分图

6.1.3. 二分图的最大匹配

6.1.3.1. 匈牙利算法

时间复杂度:$O(nm)$

https://www.luogu.com.cn/problem/B3605

给定一张左侧有 n 个点、右侧 m 个点、q 条边的二分图,求一组它的最大匹配。

```
void solve() {
    int n, m, q;
    cin >> n >> m >> q;
    vector<int> vis(n + m + 1, 0), mch(n + m + 1, 0);
    vector<vector<int>> e(n + m + 1);
    for (int i = 1; i <= q; i++) {
        int u, v;
        cin >> u >> v;
        v += n;
        e[u].push_back(v);
    }
    auto dfs = [&](auto &dfs, int u, int tag) -> bool {
        if (vis[u] == tag)
            return 0;
        vis[u] = tag;
        for (auto to : e[u])
            if (!mch[to] || dfs(dfs, mch[to], tag)) {
                mch[to] = u;
                return 1;
            }
        return 0;
    };
```

```
    int ans = 0;
    for (int i = 1; i <= n; i++)
        if (dfs(dfs, i, i))
            ans++;
    cout << ans << endl;
}
```

6.1.3.2. hk 算法

时间复杂度:$O(m\sqrt{n})$

6.1.4. 二分图最小点覆盖(König 定理)

最小点覆盖：选最少的点，满足每条边至少有一个端点被选。

二分图中，最小点覆盖 = 最大匹配。

时间复杂度:

7. 联通性

7.1. 点双/边双

7.1.1. 无向图

割点: 对于一个点 x,删去该点及所有相连的边,图不再联通

割边/桥: 对于一条边 e,删去该边,图不再联通

点双连通图:一个图不存在割点

边双连通图:一个图不存在割边

点双联通分量: 极大点双联通子图

边双联通分量: 极大边双联通子图

7.1.1.1. tarjan 求割点

若某个点是割点,一定满足以下一种情况:

1.子树中不存在跨越该点,连向该点上面的点的边

dfn[u]≤low[v]

2.该点是根且有多个儿子

若某个点是割边,一定满足:

子树中不存在跨越该点,连向该点上面的点的边

dfn[u]<low[v]

tarjan 求割点

```
int dfn[N], low[N], boo[N], cnt = 0, root;
vector<int> e[N];
void tarjan(int u)
{
    dfn[u] = low[u] = ++cnt;
    int num = 0;
    for (auto to : e[u])
        if (!dfn[to])//树边
        {
            tarjan(to);
            low[u] = min(low[u], low[to]);
            if (low[to] >= dfn[u])
            {
                num++;
                if (u != root || num > 1)
                    boo[u] = 1;
            }
        }
        else//返祖边
            low[u] = min(low[u], dfn[to]);
}
```

7.1.2. 有向图

边双:删掉桥后,每个联通块都是一个边双联通分量

点双:一个割点可能属于多个点双联通分量

用栈维护

7.2. 强连通分量

有向图:

弱联通:单向可达

强联通:双向可达

强联通分量:极大强联通子图

求强联通分量:

1.kosaraju

对任意点作为起点进行一次 dfs

按照出栈顺序存放起来,在反图上 dfs

正向联通+反向联通=强联通

```
1
```

2.tarjan

性质:每个 SCC 产生的次序是它在 DAG 中的拓扑逆序

证明:tarjan 是在搜索树中自底向上产生 SCC 的

应用:可以在 Tarjan 缩点后 DAG DP 中帮助省掉一大堆代码。

https://www.luogu.com.cn/problem/P3387

给定一个 n 个点 m 条边有向图，每个点有一个权值，求一条路径，使路径经过的点权值之和最大。你只需要求出这个权值和。

允许多次经过一条边或者一个点，但是，重复经过的点，权值只计算一次。

```cpp
#include <bits/stdc++.h>
#define x first
#define y second
#define endl '\n'
using namespace std;
using ll = long long;
using pii = pair<ll, ll>;
using db = long double;
const int N = 1e3 + 100;
const ll mod = 998244353;
void solve() {
    int n, m, idx = 0, cnt = 0;
    cin >> n >> m;
    vector<int> a(n + 1), w(n + 1, 0), ans(n + 1, 0);
    vector<int> vis(n + 1, 0), dfn(n + 1, 0), low(n + 1), bel(n + 1);
    vector<vector<int>> e(n + 1), ne(n + 1);
    for (int i = 1; i <= n; i++)
        cin >> a[i];
    for (int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;
        e[u].push_back(v);
    }
```

```

    vector<int> stk;
    auto tarjan = [&](auto &tarjan, int u) -> void {
        dfn[u] = low[u] = ++idx;
        stk.push_back(u);
        vis[u] = 1;
        for (auto &to : e[u]) {
            if (!dfn[to]) {
                tarjan(tarjan, to);
                low[u] = min(low[u], low[to]);
            } else {
                if (vis[to])
                    low[u] = min(low[u], dfn[to]);
            }
        }
        if (dfn[u] == low[u]) {
            cnt++;
            while (1) {
                auto t = stk.back();
                stk.pop_back();
                vis[t] = 0;
                bel[t] = cnt;
                w[cnt] += a[t];
                if (t == u)
                    break;
            }
        }
    };

    for (int i = 1; i <= n; i++)
        if (!dfn[i]) // make scc
            tarjan(tarjan, i);
    for (int i = 1; i <= n; i++) // make edge on scc
        for (auto &to : e[i])
            if (bel[to] != bel[i])
                ne[bel[to]].push_back(bel[i]);

    for (int i = 1; i <= cnt; i++) {
        ans[i] += w[i];
```

```
        for (auto &to : ne[i])
            if (to > i)
                ans[to] = max(ans[to], ans[i]);
    }
    int res = 0;
    for (int i = 1; i <= cnt; i++)
        res = max(res, ans[i]);
    cout << res << endl;
}
int main() {
#ifndef ONLINE_JUDGE
    freopen("in.txt", "r", stdin);
    freopen("out.txt", "w", stdout);
#endif
    // cout << fixed << setprecision(14);
    ios::sync_with_stdio(false);
    cin.tie(0), cout.tie(0);
    int _ = 1;
    // cin >> _;
    while (_--)
        solve();
    return 0;
}
```

8. 网络流

8.1. 最大流

Ford-Fulkerson 增广

Ford-Fulkerson 增广是计算最大流的一类算法的总称。该方法运用贪心的思想，通过寻找增广路来更新并求解最大流。

最大流中需要快速访问反向边,在邻接矩阵中是简单的,但如何在邻接表中实现呢?

令边从偶数开始编号,并且在加边时总是紧接着加入其反向边使得它们的编号相邻,即:

边的编号为i,反向边的编号为$i \oplus 1$

8.1.1. Edmonds-Karp 算法

时间复杂度: $O\left(|V||E|^2\right)$

每次通过 bfs 寻找增光路,直到增广路不存在,流量不会增加,算法停止

```
struct edge
{
    int u, v, c, f;  from to capacity flow
    edge(int u, int v, int c, int f) : u(u), v(v), c(c), f(f) {}
};
struct EK
{
    int n, m;
    vector<edge> e;
    vector<int> g[N];
    int pre[N], p[N];
    void init(int n)
    {
        m = 0;
        for (int i = 0; i <= n; i++)
            g[i].clear();
        e.clear();
    }
    void add(int u, int v, int cap)
    {
        e.push_back(edge(u, v, cap, 0));
        e.push_back(edge(v, u, 0, 0));
        m++;
        g[u].push_back(m - 2);
        g[v].push_back(m - 1);
    }
    int maxflow(int s, int t)
    {
        int flow = 0;
        while (1)
        {
            memset(pre, 0, sizeof pre);
            queue<int> q;
            q.push(s);
            while (q.size())
            {
                auto t = q.front();
                q.pop();
                for (auto i : g[t])
```

```
                {
                    auto [u, to, c, f] = e[i];
                    if (!pre[to] && c > f) // 这次 bfs 中还没有被访问过并且还能流
                    {
                        p[to] = i;
                        pre[to] = min(pre[t], c - f);
                        q.push(to);
                    }
                }
                if (pre[t])  // 找到了一条可行流
                    break;
            }
            if (!pre[t]) // 找不到了,break
                break;
            for (int u = t; u != s; u = e[p[u]].u)
            {
                e[p[u]].f += pre[t];    // 加上流
                e[p[u] ^ 1].f -= pre[t];  // 反向减去流
            }
            flow += pre[t];
        }
        return flow;
    }
};
```

8.1.2. Dinic 算法

时间复杂度:$O\left(|V|^2|E|\right)$

单轮增广$O(|V||E|)$,增广轮数$O(|V|)$

Dinic 算法和 EK 的最大的不同点在于每次可以增广多条路,而同时增广多条路只需要需要一个 dfs

```
struct Dinic
{
    struct edge
    {
        int to, cap, re;//to ,容量,反向边在 e 中的位置
        edge(int to, int cap, int re) : to(to), cap(cap), re(re) {}
```

```
    };
    int n, S, T;
    vector<vector<edge>> e;
    vector<int> dep, cur;//深度,弧优化
    Dinic(int n, int S, int T) : n(n), S(S), T(T)
    {
        e.resize(n + 1);
    }
    void add(int u, int v, int c)
    {
        e[u].push_back({v, c, (int)e[v].size()});
        e[v].push_back({u, 0, (int)e[u].size() - 1});
    }
    bool bfs()
    {
        cur = dep = vector<int>(n + 1);
        dep[S] = 1;
        queue<int> q;
        q.push(S);
        while (q.size())
        {
            auto u = q.front();
            q.pop();
            for (auto &[to, cap, re] : e[u])
            {
                if (dep[to] || !cap)//访问过||容量为 0
                    continue;
                dep[to] = dep[u] + 1;
                q.push(to);
            }
        }
        return dep[T];//如果 dep[T]有标号,说明存在可行流
    }
    int dfs(int u, int in)
    {
        if (u == T)
            return in;
        int out = 0;//输出流的大小
```

```
        for (int i = cur[u]; i < e[u].size(); i++)
        {
            cur[u] = i;
            auto &[to, cap, re] = e[u][i];
            if (!cap || dep[to] != dep[u] + 1)
                continue;
            int res = dfs(to, min(in, cap));
            in -= res;
            cap -= res;
            out += res;
            e[to][re].cap += res;
            if (!in)//输入流流完了
                break;
        }
        if (!out)//没有输出流
            return dep[u] = 0;
        return out;
    };
    int max_flow()
    {
        int ans = 0;
        while (bfs())
            ans += dfs(S, 2e9);
        return ans;
    }
};
```

8.1.3. ISAP 算法

时间复杂度:

8.1.4. 模型

8.2. 费用流

保证流量最大的情况下,费用最小

8.2.1. SSP(Successive Shortest Path)算法

每次寻找单位费用最小的增广路进行增广，直到图上不存在增广路为止。

实现上只需将 EK 算法或 Dinic 算法中找增广路的过程，替换为用最短路算法寻找单位费用最小的增广路即可

注意这里最短路只能使用 spfa,因为存在负权边

时间复杂度:$O\left(|V|^2|E|^2\right)$

每次找最短路(spfa)的复杂度:$O(|V||E|)*$增广路的次数$O(|V||E|)$

9. 同余最短路

https://www.luogu.com.cn/problem/P3951

求两个互质的数 a,b 最大的,不能凑出来的数

设$a<b$

那么在 %a 意义下,所有能凑出来的数可以表示为$k*a+i(1\le i\le a-1)$

按照 i 分类,分为 0,1,...,a-1

在 %a 意义下,a 的系数就不重要了,接下来考虑 b

设 f[i]表示,%a 为 i,最小的,能凑出来的数

f[i]一定是从某个 x 转移过来的,f[i]=f[(x+b)%a]

显然 f[0]=0,那么从 0 开始跑最短路,就可以得到每个位置最小能凑出来的数

因为 f[i]=k*a+i,k 是最小的,能表达出来的,%a=i 的数

那么$(k-1)*a+i$就是最大的,无法表示出来的,%a=i 的数

每个点只有一条出边,最后整个图是一个环

最大的 f[i]就是$(a-1)*b\%a$

所以答案就是 a*b-a-b